

```

        System.out.println("in m1");
        return 10;
    }
    static{
        System.out.println("Vehicle static block");
    }
}
class Car extends Vehicle{

    static int b = m2();
    public static int m2(){
        System.out.println("in m2");
        return 10;
    }
    static{
        System.out.println("Car static block");
    }
    public static void main(String[] args){
        System.out.println("in main");
    }
}

```

The output of the above code is:

```

in m1
Vehicle static block
In m2
Car static block
in main

```

The above output clarifies one thing—that the order of identification and the execution of static members in case of inheritance will occur from top to bottom or from parent class to child class.

Example 3 (when members are static, non-static and constructor): In this example, the effect of the heap area on the Java program is explained (the heap area itself has been covered earlier in the article). All the Java objects and instance members are initialised here. The method area is about static members and the heap area is about object and non-static members. In the heap area, when the Java objects are created, only then is the instance variable identified and executed. The constructors are executed in the end.

```

class Test{
    static int a=m1();
    int b=m2();
    {
        System.out.println("instance block");
    }
    public int m2(){
        System.out.println("in m2");
        return 10;
    }
}

```

```

static {
    System.out.println("in static block");
}
public static int m1(){
    System.out.println("in m1");
    return 15;
}
public Test(){
    System.out.println("in constructor");
}
public static void main(String[] args){
    System.out.println("in main");
    Test t = new Test();
}
}

```

The output of the above code is:

```

in m1
in static block
in main
in m2
instance block
in constructor

```

The above output clarifies the following three points:

1. The static member is both identified as well as executed first.
2. Instance members (non-static variables and non-static methods) are identified and executed only if the instance is created, and the order of identification and execution will be from top to bottom.
3. Constructors are always executed last.

The Java programming language becomes platform-independent because of JVM and the nature of its byte code. Because of this, Java has scaled across multiple machines, platforms and devices, powering enterprise-class Web applications as well as consumer mobile applications. The architecture of JVM has a significant effect on a Java program. Both static and non-static members of a Java class are treated differently by JVM (examples 1 and 3). Static members of a class are identified and executed as soon as the class is loaded. Non-static members are identified and executed only if the instance of the class is created. **END** 

By: Vikas Kumar Gautam

The author is a mentor at Emrbe Information Technology (P) Ltd. His main areas of expertise include application development using Java/J2EE and Android for both Web and mobile devices. A Sun Certified Java Professional (SCJP), his interests include acquiring greater expertise in the application space by learning from the latest happenings in the industry. He can be reached at vikash_kumar@emrbe.com